CodInClub — Day 2 Live-Coding Solutions

Powered by BridgeLabz | SEM 3 Week 2 Practice Problems

Problem 1: Vowel & Consonant Counter

Scenario: A library orientation kiosk counts vowels and consonants in a submitted book title for a simple text-stats display.

Suggested Signature: `void countVowelsAndConsonants(String text)` | **Concepts:** charAt(), length(), loops, case-insensitive character comparison.

Java Solution Implementation	Sample I/O

`public class VowelConsonantCounter {`
`  public static void countVowelsAndConsonants(String text) {`
`  if (text == null) return;`
`  int vowels = 0, consonants = 0;`
`  text = text.toLowerCase();`
`  for (int i = 0; i < text.length(); i++) {`
`  char ch = text.charAt(i);`
`  if (ch >= 'a' && ch <= 'z') {`
`  if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {`
`  vowels++;`
`  } else {`
`  consonants++;`
`  }`
`  }`
`  }`
`  System.out.println("Vowels: " + vowels + " | Consonants: " + consonants);`
`  }`
`}`
	Sample Input:
	"Java Programming"
	
	Sample Output:
	Vowels: 5 | Consonants: 10

Problem 2: CSV Student Record Parser

Scenario: The T&P; team receives student registration data as CSV lines and needs a quick parser to split each line into fields and print a formatted record.

Suggested Signature: `void parseStudentRecord(String csvLine)` | **Concepts:** split(), array length validation, string concatenation, formatted output.

Java Solution Implementation	Sample I/O

`public class StudentRecordParser {`
`  public static void parseStudentRecord(String csvLine) {`
`  if (csvLine == null) { System.out.println("Invalid Record"); return; }`
`  String[] fields = csvLine.split(",");`
`  if (fields.length != 3) { System.out.println("Invalid Record"); return; }`
`  String name = fields[0].trim();`
`  String rollNo = fields[1].trim();`
`  String dept = fields[2].trim();`
`  System.out.println("Name: " + name + " | Roll No: " + rollNo + " | Dept: " + dept);`
`  }`
`}`
	Sample Input:
	"Ananya Verma,RA2211003010123,CSE"
	"Ananya Verma,CSE"
	
	Sample Output:
	Name: Ananya Verma | Roll No: RA2211003010123 | Dept: CSE
	Invalid Record

Problem 3: File Extension Validator

Scenario: An assignment-upload portal must check whether an uploaded filename has an accepted extension (pdf, docx, zip) regardless of case.

Suggested Signature: `String validateFileExtension(String filename)` | **Concepts:** lastIndexOf(), substring(), equalsIgnoreCase(), conditional logic.

Java Solution Implementation	Sample I/O

`public class FileExtensionValidator {`
`  public static String validateFileExtension(String filename) {`
`  if (filename == null || !filename.contains(".")) {`
`  return "Rejected – invalid file type";`
`  }`
`  int lastDot = filename.lastIndexOf('.');`
`  String ext = filename.substring(lastDot + 1);`
`  if (ext.equalsIgnoreCase("pdf") || ext.equalsIgnoreCase("docx") || ext.equalsIgnoreCase("zip")) {`
`  return "Accepted";`
`  }`
`  return "Rejected – invalid file type";`
`  }`
`}`
	Sample Input:
	"Assignment1.PDF"
	"notes.txt"
	
	Sample Output:
	Accepted
	Rejected — invalid file type

Problem 4: Masked Phone Number Formatter

Scenario: A student-support call center displays a partially masked version of a registered phone number on-screen for privacy.

Suggested Signature: `String maskPhoneNumber(String phone)` | **Concepts:** String length/digit validation, substring(), StringBuilder insert(), masking patterns.

Java Solution Implementation	Sample I/O

`public class PhoneMasker {`
`  public static String maskPhoneNumber(String phone) {`
`  if (phone == null || phone.length() != 10) return "Invalid phone number";`
`  for (int i = 0; i < phone.length(); i++) {`
`  if (!Character.isDigit(phone.charAt(i))) return "Invalid phone number";`
`  }`
`  String last4 = phone.substring(6);`
`  StringBuilder sb = new StringBuilder("XXXXXX");`
`  sb.insert(6, "-");`
`  sb.append(last4);`
`  return sb.toString();`
`  }`
`}`
	Sample Input:
	"9876543210"
	"98765"
	
	Sample Output:
	XXXXXX-3210
	Invalid phone number

Problem 5: Bank Transaction Reference Generator & Validator

Scenario: A fintech onboarding module needs to normalize and validate transaction reference codes (14 chars: 3 letters + 6 digits + 5 digits).

Suggested Signature: `String normalizeReference(String raw) + String validateAndFormat(String reference)` |
Concepts: trim(), substring(), concatenation, Character.isLetter()/isDigit(), StringBuilder, multi-stage validation.

Java Solution Implementation	Sample I/O

`public class BankReferenceValidator {`
`  public static String normalizeReference(String raw) {`
`  if (raw == null) return null;`
`  String trimmed = raw.trim();`
`  if (trimmed.length() < 3) return trimmed;`
`  return trimmed.substring(0, 3).toUpperCase() + trimmed.substring(3);`
`  }`
`  public static String validateAndFormat(String reference) {`
`  reference = normalizeReference(reference);`
`  if (reference == null || reference.length() != 14) return "Invalid: wrong length";`
`  for (int i = 0; i < 3; i++) {`
`  if (!Character.isLetter(reference.charAt(i))) return "Invalid: bank code must be 3 letters";`
`  }`
`  for (int i = 3; i < 14; i++) {`
`  if (!Character.isDigit(reference.charAt(i))) return "Invalid: non-digit body";`
`  }`
`  String bank = reference.substring(0, 3);`
`  String dd = reference.substring(3, 5), mm = reference.substring(5, 7), yy = reference.substring(7, 9);`
`  String seq = reference.substring(9, 14);`
`  return "[" + bank + "] DATE: " + dd + "/" + mm + "/" + yy + " | SEQ: " + seq;`
`  }`
`}`
	Sample Input:
	" hdf03022600042 "
	"12F03022600042"
	
	Sample Output:
	[HDF] DATE: 03/02/26 | SEQ: 00042
	Invalid: bank code must be 3 letters